

assignment4

DSA Assignment #4: 线性结构

Updated 2026-03-30 15:22 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E160.相交链表

hash table, linked list, two pointers, <https://leetcode.cn/problems/intersection-of-two-linked-lists/>

思路:

从A, B两个头指针出发，同步往下跳，如果 `skipA == skipB` 那么指针会相遇，否则 `skip` 值小的会先走到尾节点 `None`，为了弥补这个差值，走到尾节点以后跳到对方的头指针，重复走一遍，`skip`的差值就抵消了，会同时走到交叉节点。

也可以用hash table存储链表A中的节点，枚举B中第一个出现的节点。

代码:

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
```

```
#         self.val = x
#         self.next = None

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) ->
Optional[ListNode]:
    p, q = headA, headB
    while p is not q:
        p = p.next if p else headB
        q = q.next if q else headA
    return p
```

代码运行截图 (至少包含有"Accepted")



E206.反转链表

recursion, linked list, <https://leetcode.cn/problems/reverse-linked-list/>

思路：

使用pre, cur, 初始赋值为None, head, 记录下一个指针nxt = cur.next, 然后依次赋值cur.next <- pre, pre <- cur, cur <- nxt

代码：

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        pre, cur = None, head
        while cur:
            nxt = cur.next
            cur.next = pre
            pre = cur
            cur = nxt
        return pre
```

代码运行截图（至少包含有"Accepted"）

全部提交记录

通过 28 / 28 个通过的测试用例

Jolly Turing00u 提交于 2026.03.24 17:57

分析 写题解

执行用时分布

0 ms | 击败 100.00%

消耗内存分布

20.24 MB | 击败 55.90%

代码 | Python3

```
1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
```

Python3 智能模式

```
1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def reverseList(self, head: Optional[ListNode]) -> Op
8         pre, cur = None, head
9         while cur:
10             nxt = cur.next
11             cur.next = pre
12             pre = cur
13             cur = nxt
14         return pre
```

已存储

测试用例 | 测试结果

通过 执行用时: 0 ms

Case 1 Case 2 Case 3

输入

M234.回文链表

linked list, two pointers, <https://leetcode.cn/problems/palindrome-linked-list/>

请用快慢指针实现 $O(1)$ 空间复杂度。

思路：

用快慢指针找到中间位置，然后比较前一半和后一半的反转是否相同。

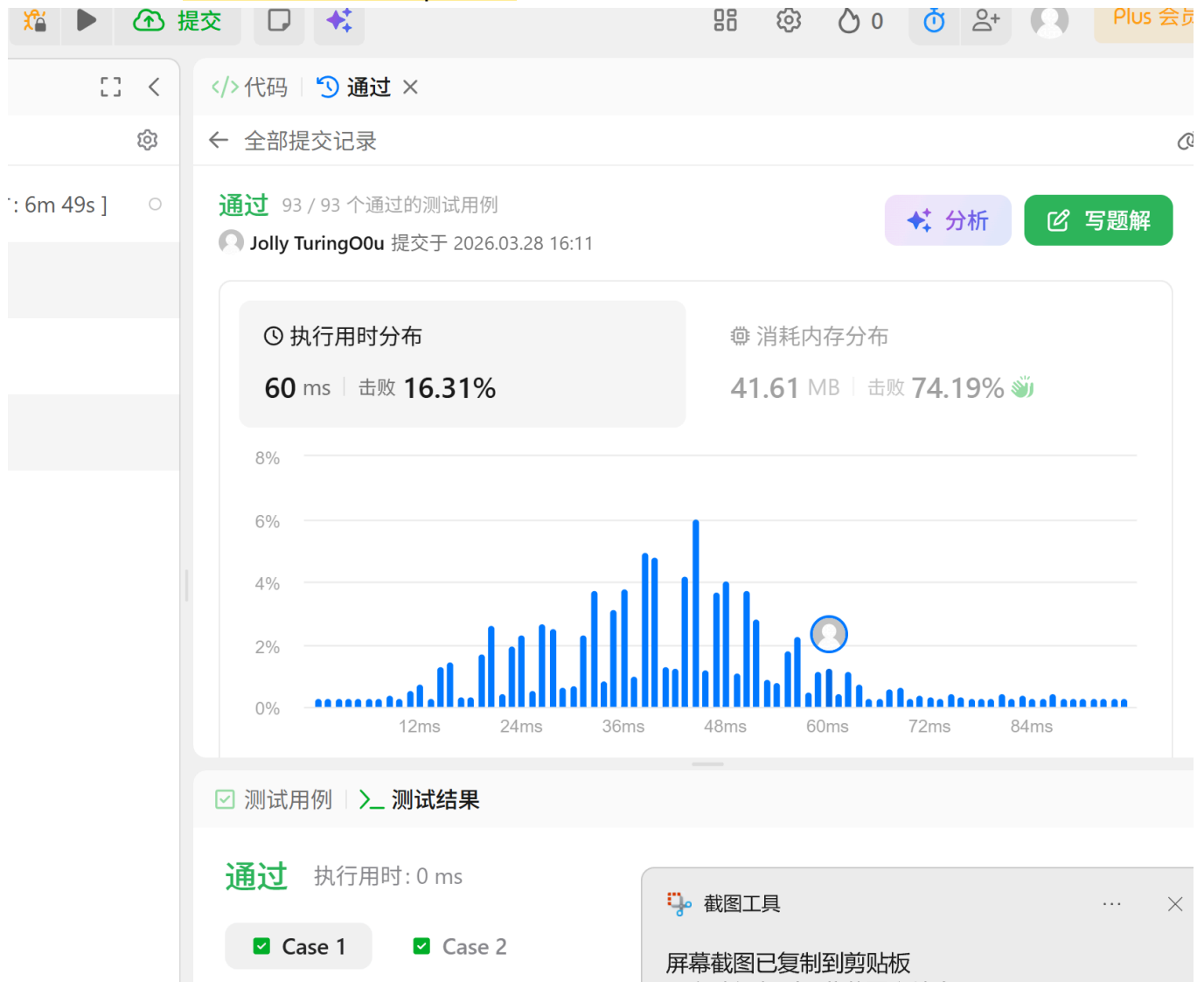
代码：

```
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        def reverse(cur):
            pre = None
            while cur:
                nxt = cur.next
                cur.next = pre
                pre = cur
                cur = nxt
            return pre

        fast = slow = head
        while fast and fast.next:
            fast = fast.next.next
            slow = slow.next

        fast = head
        slow = reverse(slow)
        while slow and fast:
            if slow.val != fast.val: return False
            slow = slow.next
            fast = fast.next
        return True
```

代码运行截图 (至少包含有"Accepted")



M24591:中序表达式转后序表达式

stack, <http://cs101.openjudge.cn/practice/24591/>

思路:

Shunting Yard 算法:

优先级管理: $*$, $/$ > $+$, $-$ > $($ 。

处理规则:

1. 操作数直接输出。
2. 遇到 $($ 入栈。
3. 遇到 $)$ 弹出栈顶直至遇到 $($ 。
4. 遇到运算符, 弹出栈中优先级更高或相等的运算符, 再将当前操作符入栈。

接受浮点数: 用 num 缓存已出现的数字 (整数&小数) 部分, 每次遇到数字 char 加入; 遇到运算符时再处理已出现的数字 num。

代码:

```
def infix_to_postfix(infix):
    prec = {'*': 2, '/': 2, '+': 1, '-': 1, '(': 0}
    opstack = []
    postfix = []
    num = ''
    for char in infix:
        if char in '0123456789.':
            num = num + char
        else:
            if num:
                postfix.append(num)
                num = ''
            if char == '(':
                opstack.append(char)
            elif char == ')':
                while opstack[-1] != '(':
                    postfix.append(opstack.pop())
                opstack.pop()
            else:
                while opstack and prec[opstack[-1]] >= prec[char]:
                    postfix.append(opstack.pop())
                opstack.append(char)
    if num:
        postfix.append(num)
    if opstack:
        postfix.extend(opstack[::-1])
    print(' '.join(postfix))

def main():
    n = int(input())
    for _ in range(n):
        infix_to_postfix(input().strip())

if __name__ == '__main__':
    main()
```

代码运行截图（至少包含有"Accepted"）

#52086999提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
def infix_to_postfix(infix):
    prec = {'*': 2, '/': 2, '+': 1, '-': 1, '(': 0}
    opstack = []
    postfix = []
    num = ''
    for char in infix:
        if char in '0123456789.':
            num = num + char
        else:
            if num:
                postfix.append(num)
                num = ''
            if char == '(':
                opstack.append(char)
            elif char == ')':
                while opstack[-1] != '(':
                    postfix.append(opstack.pop())
                opstack.pop()
```

基本信息

#: 52086999
题目: 24591
提交人: 25n2500011619
内存: 3692kB
时间: 29ms
语言: Python3
提交时间: 2026-03-24 20:36:28

M146.LRU缓存

hash table, doubly-linked list, <https://leetcode.cn/problems/lru-cache/>

思路:

首先设置 **dummy** 头节点和尾节点。当 **get** 元素时，如果存在就返回值，并将该节点移到头节点；如果不存在就返回-1。当 **put** 元素时，如果存在就值原地修改，并将该节点移到头节点；如果不存在就在头节点直接插入这个新节点，并判断是否超过了容量，如果超过了就把尾节点删掉，注意要同时把字典里面的这个键值对删去，可以 `del dict[key]` 或 `dict.pop(key)`。

把节点移到头部分为两步：第一步，获得这个节点的 **val** 值，把它当作一个新节点插入头部；第二步，在链表中删除原来节点。因此只需要两个辅助函数。

代码

```
class Node:
    __slots__ = 'prev', 'next', 'key', 'value'
    def __init__(self, key = -1, value = -1):
        self.key = key
        self.value = value # 节点数据
        self.next = None # 指向下一个节点
        self.prev = None # 指向前一个节点

class LRUCache:

    def __init__(self, capacity: int):
        self.n = capacity
        self.head = Node()
```

```

self.head.next = self.head
self.head.prev = self.head
self.dic = {}

def push_front(self, cur):
    cur.prev = self.head
    cur.next = self.head.next
    cur.prev.next = cur
    cur.next.prev = cur

def remove(self, cur):
    cur.prev.next = cur.next
    cur.next.prev = cur.prev

def get(self, key: int) -> int:
    if key in self.dic:
        cur = self.dic[key]
        self.remove(cur)
        self.push_front(cur)
        return cur.value
    return -1

def put(self, key: int, value: int) -> None:
    if key in self.dic:
        cur = self.dic[key]
        self.remove(cur)
        cur.value = value
        self.push_front(cur)
    else:
        new = Node(key, value)
        self.push_front(new)
        if self.n > 0: self.n -= 1
        else:
            self.dic.pop(self.head.prev.key)
            self.remove(self.head.prev)
        self.dic[key] = new

```

```

# Your LRUCache object will be instantiated and called as such:
# obj = LRUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)

```

(至少包含有"Accepted")



P2698 [USACO12MAR] Flowerpot S

monotonic queue, <https://www.luogu.com.cn/problem/P2698>

思路:

要找一个坐标 y 之差大于等于 d 的最小 x 区间, 可以采用二分区间长度 + $O(\max\{x\})$ 判断的方法, 也可以先用单调栈, 再二分查找。

具体思路是维护一个单调递增栈, 如果栈顶高度 $\geq y$ 就弹出; 找第一个小于等于 $y-d$ 的 x 位置, 然后反向做一遍, 时间复杂度 $O(n \log n)$ 。

代码

```
def min_width(pos, d):
    stack = []
    ans = float('inf')
    for x, y in pos:
        while stack and stack[-1][1] >= y:
            stack.pop()
        if stack:
            l, r = 0, len(stack) - 1
            while l <= r:
```

```

        mid = (l + r) >> 1
        if stack[mid][1] <= y - d:
            l = mid + 1
            ans = min(ans, abs(x - stack[mid][0]))
        else: r = mid - 1
        stack.append((x, y))
    return ans

def main():
    n, d = map(int, input().split())
    pos = [tuple(map(int, input().split())) for _ in range(n)]
    pos.sort()
    ans = min(min_width(pos, d), min_width(pos[::-1], d))
    print(ans if ans != float('inf') else -1)

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

编程语言	代码长度	用时	内存
Python 3	747B	1.43s	18.02MB

[测试点信息](#)
[源代码](#)

测试点信息

#1 AC 20ms/3.90MB	#2 AC 21ms/3.88MB	#3 AC 21ms/4.13MB	#4 AC 23ms/4.16MB	#5 AC 36ms/4.61MB	#6 AC 89ms/6.82MB	#7 AC 169ms/9.54MB
#8 AC 253ms/12.32MB	#9 AC 346ms/15.21MB	#10 AC 454ms/18.02MB				

Q_jan

所属题目

P2698 [USACO12MAR] Flowerp...

评测状态

Accepted

提交时间

2026-03-24 21:11:59

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

作业里的相交、反向、回文、双向链表在刷LC热题100的时候做过，这次重新复习了一遍。学习了中缀表达式转后序表达式的Shunting Yard算法，它的核心思想就像列车调度，所以称调度场算法。

第一部分：Shunting Yard 算法（中缀转后缀 Postfix）

想象一个火车站（调度场），有一列火车（你的中缀表达式）正开过来。车厢有两种：数字（操作数）和符号（运算符/括号）。

- **数字**：是 VIP 乘客，不需要排队等待，直接走向“出口”(输出结果)。
- **符号**：需要根据“优先级”进入一个“候车室”(栈 Stack) 等待。优先级高的符号可以先进站，如果优先级相等，后进入的符号先出站，或者如果遇到括号，括号里的符号要优先处理。

第二部分：中缀转前缀表达式 (Prefix)

前缀表达式（波兰式）的特点是运算符在操作数的前面。把中缀转前缀，最简单的方法其实就是“反向的 Shunting Yard”。

算法步骤：

1. **反转输入**：把中缀表达式从右向左读（也就是完全反转）。注意：反转时，原来的右括号 `)` 会变成左括号 `(`，原来的左括号 `(` 会变成右括号 `)`。
2. **执行微调版的 Shunting Yard**：
 - 数字：依然直接输出。
 - 遇到左括号（原来的右括号）：压入栈。
 - 遇到右括号（原来的左括号）：持续出栈并输出，直到遇到左括号。
 - **遇到运算符**：这里的规则有一点小变化。如果栈顶运算符的优先级 **严格大于**（而不是大于等于）当前运算符，栈顶才出栈。这是为了保持结合律的正确性。然后当前运算符入栈。
3. **反转输出**：把第2步得到的输出结果再反转一次，就是最终的前缀表达式！

这周额外做了每日选作里的3327. 判断 DFS 字符串是否是回文串，1888. 使二进制字符串字符交替的最少反转次数，练习了在后序遍历串中跑马拉车，滑动窗口结合位运算，这些基本的算法组合起来以后会大大提高难度，对算法的熟练度要求更高。